

FULL-STACK CAPSTONE PROJECT DEFENSE

PathFinder Nexus

Autonomous, Dependency-Aware & Continuously Adaptive Learning Navigator

FastAPI (Python 3.12) • PostgreSQL 16 & pgvector • React 18 SPA • Google Gemini LLM

162 Backend Unit & Integration Tests (100% Deterministic Fallback)

Topological Kahn's DAG Engine • 6-Factor Recommendation • Bayesian Evidence Fusion

Why Modern Online Education Fails Learners

01

Linear, Static Playlists

Courses assume identical starting baselines, forcing experts through basics and abandoning beginners when advanced prerequisites are skipped.

02

Hallucinating AI Chatbots

Generic LLMs invent fake URLs, fabricate invalid course sequences, and lack relational database grounding and dependency validation.

03

Zero Adaptive Feedback

When learners fail quizzes, static platforms do not dynamically restructure road maps or deliver prerequisite refresher interventions.

The PathFinder Nexus Architectural Creed

“RAG retrieves. LLMs explain. The Database grounds. Deterministic engines decide.”

A strict architectural separation of concerns preventing AI hallucinations and ensuring mathematical integrity.

RAG (pgvector)

Retrieves verified educational resources via 1536-dim cosine similarity.

LLM (Gemini)

Synthesizes clear explanations, goal summaries, and interactive tutor answers.

Database (Postgres)

Authoritative single source of truth for roles, skills, questions, and roadmaps.

Deterministic Engines

Topological Kahn DAG, 6-factor ranking, and Bayesian evidence fusion.

Modular Monolith Architecture & Technology Stack

1. Frontend Client (SPA)

- React 18 + TypeScript + Vite
- Tailwind CSS Dark Theme & Glassmorphism
- Client-Side JWT Auth with persistence
- Netlify Edge CDN Deployment

2. FastAPI Backend Core

- Python 3.12 Modular Monolith
- Clean Architecture (Routers/Services/Repos)
- Argon2id Password Security + Correlation IDs
- Render Cloud Web Service (Uvicorn)

3. Data & AI Layer

- PostgreSQL 16 with pgvector Extension
- 22 Normalized Relational Entity Tables
- Google Gemini 1.5/2.0 Flash + OpenAI Embeddings
- Deterministic Mock Fallback Provider

AI Goal Understanding & Grounded Extraction

Natural Language Input:

"I want to become an AI Engineer specializing in LLMs & RAG systems within 12 weeks with 2 hours daily study."

Structured Pydantic Extraction:

- Target Role: AI/ML Engineer (Matched)
- Target Duration: 12 Weeks
- Daily Study: 2.0 Hours
- Extracted Skills: Python (40%), LLMs (0%), Vector DBs (0%)

Canonical Role Grounding Pipeline:

1. Raw Prompt Sanitization & XML Delimiter Tagging
2. Gemini Structured JSON Mode with Pydantic Validation
3. Database Lookup: Matches against 10 Canonical Roles
4. Ambiguity Handler: Returns suggested roles if unresolved
5. Zero Hallucinations: Non-existent roles never persisted

Dynamic Skill Gap & Role Readiness Engine

Zero Static Storage

Skill gaps are computed on-the-fly dynamically upon query. Eliminates stale cached tables

Gap Formulation

$\text{Gap} = \max(\text{Required} - \text{LearnerProficiency}, 0)$

Strictly non-negative delta for every skill required by role.

Role Readiness Score

$\text{Readiness} = (\sum \text{Current_w} / \sum \text{Required_w}) \times 100$

Weighted importance factor ensures critical skills drive readiness.

Topological Roadmap Engine (Kahn's DAG Algorithm)

Prerequisite-Aware Directed Acyclic Graph

- Strict in-degree topological sort
- Guarantees prerequisites precede advanced topics
- Cycle detection prevents circular dependencies
- Dynamic milestone unlocking based on mastery

Kahn's Execution Workflow:

1. Calculate in-degree for all required role skills
2. Initialize queue with all in-degree == 0 nodes
3. Pop node u -> append to roadmap sequence
4. Decrement in-degree for outgoing neighbors v
5. If $\text{in-degree}[v] == 0$, push v to queue
6. Assign milestone sequence & estimated weeks

Normalized 6-Factor Explainable Recommendation

$$\text{Score} = 0.30(\text{Gap}) + 0.20(\text{Prereq}) + 0.15(\text{Goal}) + 0.15(\text{Diff}) + 0.10(\text{Time}) + 0.10(\text{Pref})$$

Multi-objective normalized scoring where all weight coefficients sum strictly to 1.0.

30% — Skill Gap Weight

Prioritizes biggest skill gaps

20% — Prereq Readiness

Ensures learner is ready

15% — Goal Alignment

Matches target role curriculum

15% — Difficulty Match

Aligns with user level

10% — Time Investment

Fits daily study budget

10% — Format Preference

Video, article, code balance

Sanitized Assessments & Bayesian Evidence Fusion

Anti-Cheat Sanitized Delivery

- Correct answers NEVER sent to browser client
- Client receives only question IDs & option keys
- 100% Authoritative Server-Side Grading
- Eliminates browser inspect element cheat exploits

Bayesian Evidence Fusion Formula:

$$P_{\text{new}} = \text{round}(0.30 \times P_{\text{old}} + 0.70 \times \text{Score}, 2)$$

- Prevents single-test score volatility
- Mastery Thresholds:
 - Mastered: $\text{Score} \geq 80\%$
 - In Progress: $60\% \leq \text{Score} < 80\%$
 - Critical Deficiency: $\text{Score} < 60\%$

Closed-Loop Adaptive Interventions & Next Best Action

Score < 40%

Foundational Lock

Locks dependent roadmap items, inserts prerequisite refresher resources, and updates Next Best Action.

40% ≤ Score < 80%

Targeted Reinforce

Delivers focused practice problems while keeping milestone active for revision.

Score ≥ 80%

Milestone Mastery

Marks milestone completed, updates topological graph, and unlocks downstream milestones.

Grounded RAG Knowledge Assistant & Citation Safety

pgvector Semantic Search

- 1536-dimensional OpenAI vector embeddings
- Cosine similarity threshold cutoff (≥ 0.50)
- Top-K semantic context retrieval from Postgres

Anti-Hallucination Guardrails

- Strict XML security delimiters protect context
- Every answer cites validated database resource URLs
- Zero-context fallback refuses to fabricate URLs

Relational Database Schema (22 Core Tables)

Auth & Users

- users
- learner_profiles
- learner_skills
- user_preferences

Catalog & Graph

- roles
- skills
- skill_prerequisites
- role_skills

Roadmaps & Content

- roadmaps
- roadmap_items
- resources
- projects

Assess & Chat

- assessments
- assessment_questions
- submissions
- chat_messages

Automated Testing & Benchmark Results

162 Tests

Backend Pytest Suite

100% passing across auth, DAG roadmaps, recommendations, evidence fusion, and security.

0 Errors

TypeScript Strict Build

Zero TypeScript compilation errors with optimized Vite production bundle.

100% Safe

Deterministic Fallback

System operates smoothly even when cloud AI rate limits hit or in offline mode.

Production Cloud Deployment Architecture

Frontend: Netlify

- React 18 SPA on Global Edge CDN
- Automated netlify.toml headers & caching
- _redirects file for SPA client-side deep linking
- Command: npm run build

Backend: Render

- FastAPI Web Service running Uvicorn
- render.yaml blueprint configuration
- Dynamic CORS regex matching Netlify previews
- Health Check Probes: /health & /health/ready

Database: PostgreSQL

- Cloud PostgreSQL 16 Instance
- Automated migrations via deploy_init.py
- Idempotent catalog seeding on startup
- pgvector similarity indexing

CONCLUSION & PROJECT DEFENSE

PathFinder Nexus

“Transforming natural-language ambition into deterministic, adaptive mastery.”

✓ 7 Fully Integrated Engines | ✓ 162 Passed Tests | ✓ Production Deployment-Ready

Thank you! We are ready for live demonstration & questions.